

Topic 2 – Machine Learning: Linear Regression

Cell-by-Cell Explanation of the Uploaded Jupyter Notebook

Source notebook: **Topic 2-Machine Learning - Linear Regression.ipynb**

The explanation below follows the notebook exactly in its original cell order. Notebook outputs are interpreted where outputs were saved in the file; no missing results are invented.

Notebook item	Value
Total cells	38
Dataset	Scikit-learn Diabetes Dataset
Rows	442
Columns	11 total: 10 features + target
Selected feature	bmi
Train/Test split	80% / 20%
Train/Test rows	353 / 89
Model	<code>sklearn.linear_model.LinearRegression</code>
Test R ²	26.66%

1. Machine Learning Workflow in This Notebook

Step	What the notebook does
1. Load data	Loads the Diabetes dataset from scikit-learn.
2. Build DataFrame	Combines the ten features and target into `df`.
3. EDA	Checks dtypes, missing values, descriptive statistics, and duplicates.
4. Visualization	Plots the target distribution and correlation heatmap.
5. Feature selection	Ranks correlation with target and selects `bmi`.
6. Preprocessing	Creates a median-imputation pipeline.
7. Split data	Uses 80% training and 20% testing data.
8. Train model	Fits `LinearRegression` on BMI.
9. Predict	Predicts target values for the test set.
10. Evaluate	Calculates MSE, RMSE, MAE, and R ² .

Core idea: the notebook uses a **simple linear regression** model because only one feature, `bmi`, is passed to the final model. Scikit-learn describes LinearRegression as ordinary least squares and stores learned coefficients and the intercept in the fitted estimator. [cite turn0search0](#)

Cell 1 – Notebook question / setup

Cell type: markdown

Original notebook content

Q1. Dataset Loading

****Load the Diabetes Dataset using Scikit-learn and display the first 10 records, dataset shape, feature names, and target variable.**

```
from sklearn.datasets import load_diabetes

df_X, series_y = load_diabetes(as_frame=True, return_X_y=True)
y = pd.DataFrame([series_y]).T
```

What this cell does

This markdown cell states the task: load the Diabetes dataset from scikit-learn and inspect the records, shape, feature names, and target. The two lines shown below the question are also present inside this markdown cell, so they are treated as part of the notebook's original content.

ML concept / developer understanding

The overall goal is to prepare a regression dataset where the target is a continuous diabetes disease-progression measure.

Cell 2 – Import libraries and load data

Cell type: code

Original notebook content

```
import pandas as pd
from sklearn.datasets import load_diabetes

df_X, series_y = load_diabetes(as_frame=True, return_X_y=True)
y = pd.DataFrame([series_y]).T
```

What this cell does

`pandas` is imported for DataFrame operations. `load\_diabetes` loads the built-in Diabetes dataset. With `as\_frame=True` and `return\_X\_y=True`, scikit-learn returns the features as a pandas DataFrame and the target as a pandas Series. The target is then converted to a one-column DataFrame named `y`.

ML concept / developer understanding

This establishes X (features) and y (target), the standard supervised-learning structure.

Cell 3 – Create the combined DataFrame

Cell type: code

Original notebook content

```
df = pd.DataFrame(df_X)
df['target'] = series_y
```

What this cell does

`df` is created from the feature DataFrame. A new `target` column is added from `series\_y`, producing one table containing all predictors plus the target.

ML concept / developer understanding

Keeping features and target together makes EDA and correlation analysis easier.

Cell 4 – View first 10 records

Cell type: code

Original notebook content

```
df.head(10)
```

What this cell does

`df.head(10)` displays the first ten rows. The stored output confirms the dataset contains the expected scaled-looking numeric values.

ML concept / developer understanding

This is a quick sanity check that the data loaded correctly and the columns look reasonable.

Stored notebook output

```
age      sex      bmi      bp      s1      s2      s3      \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142
5 -0.092695 -0.044642 -0.040696 -0.019442 -0.068991 -0.079288  0.041277
6 -0.045472  0.050680 -0.047163 -0.015999 -0.040096 -0.024800  0.000779
7  0.063504  0.050680 -0.001895  0.066629  0.090620  0.108914  0.022869
8  0.041708  0.050680  0.061696 -0.040099 -0.013953  0.006202 -0.028674
9 -0.070900 -0.044642  0.039062 -0.033213 -0.012577 -0.034508 -0.024993

      s4      s5      s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
5 -0.076395 -0.041176 -0.096346    97.0
6 -0.039493 -0.062917 -0.038357   138.0
7  0.017703 -0.035816  0.003064    63.0
8 -0.002592 -0.014960  0.011349   110.0
9 -0.002592  0.067737 -0.013504   310.0
```

Cell 5 – Check dataset shape

Cell type: code

Original notebook content

```
# display shape
df.shape
```

What this cell does

`df.shape` returns `(442, 11)`: 442 rows and 11 columns. Ten columns are features and one column is the target.

ML concept / developer understanding

Shape checking is important before modeling because it confirms the number of observations and variables.

Stored notebook output

```
(442, 11)
```

Cell 6 – Inspect DataFrame information

Cell type: code

Original notebook content

```
#display Info
df.info()
```

What this cell does

`df.info()` shows 442 entries and 11 columns. The stored output shows all columns are non-null and numeric (`float64`).

ML concept / developer understanding

This checks data types, row counts, and missing-value presence at a structural level.

Stored notebook output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
#   Column  Non-Null Count  Dtype
---  -
0    age      442 non-null     float64
1    sex      442 non-null     float64
2    bmi      442 non-null     float64
3    bp       442 non-null     float64
4    s1       442 non-null     float64
5    s2       442 non-null     float64
6    s3       442 non-null     float64
7    s4       442 non-null     float64
8    s5       442 non-null     float64
9    s6       442 non-null     float64
10   target   442 non-null     float64
dtypes: float64(11)
memory usage: 38.1 KB
```

Cell 7 – List feature names

Cell type: code

Original notebook content

```
df_X.columns.tolist()
```

What this cell does

`df_X.columns.tolist()` converts the feature-column index into a normal Python list. The notebook shows ten features: age, sex, bmi, bp, s1, s2, s3, s4, s5, and s6.

ML concept / developer understanding

Feature names are useful when selecting predictors and interpreting the model.

Stored notebook output

```
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
```

Cell 8 – EDA section heading

Cell type: markdown

Original notebook content

```
**Q2. Exploratory Data Analysis (EDA)**
```

What this cell does

This markdown cell marks the beginning of Exploratory Data Analysis. No computation is performed here.

ML concept / developer understanding

EDA helps understand data quality, distributions, and relationships before model training.

Cell 9 – Check data types

Cell type: code

Original notebook content

```
df.dtypes
```

What this cell does

`df.dtypes` reports the dtype of every column. The stored output shows all 11 columns are `float64`.

ML concept / developer understanding

Numeric dtypes are appropriate for the correlation and LinearRegression steps used later.

Stored notebook output

```
age      float64
sex      float64
bmi      float64
bp       float64
s1       float64
s2       float64
s3       float64
s4       float64
s5       float64
s6       float64
target   float64
dtype: object
```

Cell 10 – Find missing values

Cell type: code

Original notebook content

```
# to find missing value
df.isna().sum()
```

What this cell does

`df.isna().sum()` counts missing values column by column. The stored output is zero for every column.

ML concept / developer understanding

Missing-value checks prevent unexpected failures and help decide whether imputation is required.

Stored notebook output

```
age      0
sex      0
bmi      0
bp       0
s1       0
s2       0
s3       0
s4       0
s5       0
s6       0
target   0
dtype: int64
```

Cell 11 – Statistical summary

Cell type: code

Original notebook content

```
# to describe
df.describe()
```

What this cell does

`df.describe()` calculates count, mean, standard deviation, minimum, quartiles, and maximum for numeric columns. The notebook output confirms 442 observations for each variable.

ML concept / developer understanding

This gives a statistical overview and helps identify unusual ranges or possible outliers.

Stored notebook output

	age	sex	bmi	bp	s1	\
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	-2.511817e-19	1.230790e-17	-2.245564e-16	-4.797570e-17	-1.381499e-17	-1.381499e-17
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02
min	-1.072256e-01	-4.464164e-02	-9.027530e-02	-1.123988e-01	-1.267807e-01	-1.267807e-01
25%	-3.729927e-02	-4.464164e-02	-3.422907e-02	-3.665608e-02	-3.424784e-02	-3.424784e-02
50%	5.383060e-03	-4.464164e-02	-7.283766e-03	-5.670422e-03	-4.320866e-03	-4.320866e-03
75%	3.807591e-02	5.068012e-02	3.124802e-02	3.564379e-02	2.835801e-02	2.835801e-02
max	1.107267e-01	5.068012e-02	1.705552e-01	1.320436e-01	1.539137e-01	1.539137e-01

	s2	s3	s4	s5	s6	\
count	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02	4.420000e+02
mean	3.918434e-17	-5.777179e-18	-9.042540e-18	9.293722e-17	1.130318e-17	1.130318e-17
std	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02	4.761905e-02
min	-1.156131e-01	-1.023071e-01	-7.639450e-02	-1.260971e-01	-1.377672e-01	-1.377672e-01
25%	-3.035840e-02	-3.511716e-02	-3.949338e-02	-3.324559e-02	-3.317903e-02	-3.317903e-02
50%	-3.819065e-03	-6.584468e-03	-2.592262e-03	-1.947171e-03	-1.077698e-03	-1.077698e-03
75%	2.984439e-02	2.931150e-02	3.430886e-02	3.243232e-02	2.791705e-02	2.791705e-02
max	1.987880e-01	1.811791e-01	1.852344e-01	1.335973e-01	1.356118e-01	1.356118e-01

	target
count	442.000000
mean	152.133484
std	77.093005
min	25.000000
25%	87.000000
50%	140.500000
75%	211.500000
max	346.000000

Cell 12 – Check duplicate rows

Cell type: code

Original notebook content

```
# find duplicated value
df.duplicated().sum()
```

What this cell does

`df.duplicated().sum()` counts exact duplicate rows. The stored result is 0.

ML concept / developer understanding

Removing or investigating duplicates can prevent repeated observations from influencing a model.

Stored notebook output

```
np.int64(0)
```

Cell 13 – Visualization section heading

Cell type: markdown

Original notebook content

****Q3. Data Visualization****

What this cell does

This markdown cell marks the start of data visualization.

ML concept / developer understanding

Visual EDA helps you understand distributions and relationships that summary statistics alone may not show.

Cell 14 – Import plotting libraries

Cell type: code

Original notebook content

```
import matplotlib.pyplot as plt
import seaborn as sns
```

What this cell does

`matplotlib.pyplot` provides plotting functions and `seaborn` provides higher-level statistical plots.

ML concept / developer understanding

The notebook uses these libraries for the target distribution, correlation heatmap, and scatter plot.

Cell 15 – Plot target distribution

Cell type: code

Original notebook content

```
plt.figure(figsize=(6,4))

sns.histplot(df['target'],kde=True)

plt.title("Distribution of Diabetes Target")
plt.xlabel("Target")
plt.ylabel("Frequency")
plt.show()
```

What this cell does

`plt.figure(figsize=(6,4))` creates the plotting area. `sns.histplot(df['target'], kde=True)` plots the target distribution and adds a KDE curve. Titles and axis labels make the chart readable.

ML concept / developer understanding

A histogram helps determine how the continuous target values are distributed.

Stored notebook output

<Figure size 600x400 with 1 Axes>

Cell 16 – Histogram observation

Cell type: markdown

Original notebook content

****Observation:-****

The Histogram shows the distribution of the diabetes targets values. The values are spread across different ranges, with

What this cell does

The notebook states that target values are spread across different ranges with variation in frequency.

ML concept / developer understanding

This is the notebook's own observation; it does not claim a specific distribution such as normality.

Cell 17 – Correlation heatmap

Cell type: code

Original notebook content

```
plt.figure(figsize=(6,4))
corr = df.corr()

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Heatmap")
plt.show()
```

What this cell does

`df.corr()` calculates pairwise Pearson correlations. `sns.heatmap(..., annot=True, cmap='coolwarm', fmt='.2f')` displays those correlations with numeric annotations. The plot helps identify features that move more strongly with the target.

ML concept / developer understanding

Correlation is useful for initial feature analysis, but correlation alone does not prove causation or guarantee predictive performance.

Stored notebook output

<Figure size 600x400 with 2 Axes>

Cell 18 – Rank features by target correlation

Cell type: code

Original notebook content

```
# Find the most correlated feature
correlation_with_target = df.corr()['target'].drop('target').abs().sort_values(ascending=False)

print("Feature ranked by absolute correlation with target :")
print(correlation_with_target)
```

What this cell does

The code selects the `'target'` correlation column, removes the target's self-correlation, takes absolute values, and sorts from largest to smallest. The stored output ranks `'bmi'` first at about 0.586, followed by `'s5'` at about 0.566 and `'bp'` at about 0.441.

ML concept / developer understanding

Absolute correlation lets the notebook rank both positive and negative relationships by strength.

Stored notebook output

```
Feature ranked by absolute correlation with target :
bmi      0.586450
s5       0.565883
bp       0.441482
s4       0.430453
s3       0.394789
s6       0.382483
s1       0.212022
age      0.187889
s2       0.174054
sex      0.043062
Name: target, dtype: float64
```

Cell 19 – Select the strongest feature and plot it

Cell type: code

Original notebook content

```
# Get most correlated Features
most_corrlated_feature = correlation_with_target.index[0]

print("Most correlated features :", most_corrlated_feature)

#scatter plot
plt.figure(figsize=(6,4))

sns.scatterplot(
    x=df[most_corrlated_feature],
    y=df['target']
)

plt.title(f"{most_corrlated_feature} VS Target")
plt.xlabel(most_corrlated_feature)
plt.ylabel("Target")
plt.show()
```

What this cell does

`correlation\_with\_target.index[0]` selects the top-ranked feature, which the notebook output shows is `bmi`. A Seaborn scatter plot then compares BMI with the target.

ML concept / developer understanding

This visual check complements the numeric correlation and supports the later choice of BMI as the single predictor.

Stored notebook output

Most correlated features : bmi

<Figure size 600x400 with 1 Axes>

Cell 20 – Scatter-plot observation

Cell type: markdown

Original notebook content

****observation:****

The scatter plot shows the relationship between the most correlated feature and the target variable. The direction of the

What this cell does

The notebook says the point pattern shows the relationship between the most correlated feature and the target, and that stronger correlation generally produces a clearer trend.

ML concept / developer understanding

The statement is qualitative and matches the purpose of the scatter plot.

Cell 21 – Preprocessing section heading

Cell type: markdown

Original notebook content

Q4. Data Preprocessing

What this cell does

This markdown cell marks the start of preprocessing.

ML concept / developer understanding

Preprocessing prepares X and y for the train/test modeling workflow.

Cell 22 – Select model feature and target

Cell type: code

Original notebook content

```
X = df[['bmi']]
Y = df[['target']]
display(X)
display(Y)
```

What this cell does

`X = df[['bmi']]` selects BMI as a one-column feature DataFrame. `Y = df[['target']]` selects the target as a one-column DataFrame. `display()` shows both.

ML concept / developer understanding

The notebook intentionally builds a simple linear regression using one predictor: BMI.

Stored notebook output

```
bmi
0    0.061696
1   -0.051474
2    0.044451
3   -0.011595
4   -0.036385
..      ...
437  0.019662
438 -0.015906
439 -0.015906
440  0.039062
441 -0.073030

[442 rows x 1 columns]
target
0    151.0
1     75.0
2    141.0
3    206.0
4    135.0
..      ...
437  178.0
438  104.0
439  132.0
440  220.0
441   57.0

[442 rows x 1 columns]
```

Cell 23 – Create an imputation pipeline

Cell type: code

Original notebook content

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer

num_pipe = make_pipeline(SimpleImputer(strategy='median')).set_output(transform='pandas')
num_pipe
```

What this cell does

`SimpleImputer(strategy='median')` defines median-value imputation for missing numeric values. `make\_pipeline(...)` wraps it into a reusable pipeline, and `set\_output(transform='pandas')` asks for pandas output.

ML concept / developer understanding

The current dataset has no missing values, but the pipeline makes the preprocessing step reusable and robust if missing values appear later.

Stored notebook output

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median'))])
```

Cell 24 – Split into training and testing sets

Cell type: code

Original notebook content

```
# Splitting the data into 80% training and 20% testing sets
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest = train_test_split(X,Y,train_size=0.8,random_state=21)
```

What this cell does

`train\_test\_split` separates the data into 80% training and 20% testing sets. `random\_state=21` makes the split reproducible. The stored shapes later confirm 353 training rows and 89 testing rows.

ML concept / developer understanding

Training data is used to fit the model; unseen test data is used to estimate generalization performance.

Cell 25 – Model training section heading

Cell type: markdown

Original notebook content

```
**Q5. Model Training**
```

What this cell does

This markdown cell marks the beginning of model-training steps.

ML concept / developer understanding

The next cells inspect the split, fit preprocessing, transform the data, and train LinearRegression.

Cell 26 – Check X split sizes

Cell type: code

Original notebook content

```
print("X Training and Testing size :", xtrain.shape,xtest.shape)
```

What this cell does

The notebook prints `(353, 1)` for training X and `(89, 1)` for testing X.

ML concept / developer understanding

This confirms the 80/20 split and that there is one predictor column.

Stored notebook output

```
X Training and Testing size : (353, 1) (89, 1)
```

Cell 27 – Check y split sizes

Cell type: code

Original notebook content

```
print("Y Training and Testing size :", ytrain.shape, ytest.shape)
```

What this cell does

The notebook prints `(353, 1)` for training y and `(89, 1)` for testing y.

ML concept / developer understanding

This confirms X and y remain aligned in sample counts.

Stored notebook output

```
Y Training and Testing size : (353, 1) (89, 1)
```

Cell 28 – Fit preprocessing on training data

Cell type: code

Original notebook content

```
num_pipe.fit(xtrain)
```

What this cell does

`num\_pipe.fit(xtrain)` learns the preprocessing parameters from the training feature data. With median imputation, the relevant statistic is the training-set median.

ML concept / developer understanding

Fitting preprocessing only on training data helps avoid leaking information from the test set into training.

Stored notebook output

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median'))])
```

Cell 29 – Transform train and test data

Cell type: code

Original notebook content

```
xtrain_pre = num_pipe.transform(xtrain)
xtest_pre = num_pipe.transform(xtest)
```

What this cell does

`num\_pipe.transform(xtrain)` and `num\_pipe.transform(xtest)` apply the learned preprocessing to both datasets. The results are stored in `xtrain\_pre` and `xtest\_pre`.

ML concept / developer understanding

Both train and test data must pass through the same fitted preprocessing process before prediction.

Cell 30 – Build and train Linear Regression

Cell type: code

Original notebook content

```
# module building
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(xtrain_pre, ytrain)
```

What this cell does

`LinearRegression()` creates an ordinary least-squares linear regression estimator. `model.fit(xtrain\_pre, ytrain)` learns the intercept and coefficient from the training data.

ML concept / developer understanding

The model learns a relationship of the form $\hat{y} = \text{intercept} + \text{coefficient} \times \text{BMI}$. Scikit-learn documents `LinearRegression` as an ordinary least-squares model that minimizes residual sum of squares.

■cite■turn0search0■

Stored notebook output

```
LinearRegression()
```

Cell 31 – Read intercept and slope

Cell type: code

Original notebook content

```
print(f"Yintercept: {model.intercept_}, Slope: {model.coef_}")
```

What this cell does

The stored output gives an intercept of about 152.795 and a BMI coefficient of about 970.942. In this one-feature model, the coefficient is the estimated change in the predicted target for a one-unit increase in the BMI feature as represented in the dataset.

ML concept / developer understanding

The large-looking coefficient is partly explained by the Diabetes dataset's scaled feature representation; interpret it in the dataset's feature units rather than ordinary BMI units.

Stored notebook output

```
Yintercept: [152.79468137], Slope: [[970.94219505]]
```

Cell 32 – Prediction section heading

Cell type: markdown

Original notebook content

```
## Q6. Prediction
```

What this cell does

This markdown cell marks the prediction stage.

ML concept / developer understanding

After training, the model is applied to previously unseen test features.

Cell 33 – Generate predictions

Cell type: code

Original notebook content

```
ypred_test = model.predict(xtest_pre)
ypred_test[:5]
```

What this cell does

`model.predict(xtest\_pre)` predicts target values for all test rows. `ypred\_test[:5]` displays the first five predictions. The stored output begins with approximately 173.98, 144.68, 222.12, 166.65, and 191.77.

ML concept / developer understanding

Prediction is the model's estimated target value for each test observation.

Stored notebook output

```
array([[173.97788238],
       [144.6760722 ],
       [222.11657054],
       [166.65242983],
       [191.76826713]])
```

Cell 34 – Model evaluation section heading

Cell type: markdown

Original notebook content

****Q7. Model Evaluation****

What this cell does

This markdown cell marks the evaluation stage.

ML concept / developer understanding

The notebook evaluates prediction error using MSE, RMSE, MAE, and R².

Cell 35 – Calculate regression metrics

Cell type: code

Original notebook content

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(ytest, ypred_test)
rmse = mse**(1/2)
mae = mean_absolute_error(ytest, ypred_test)
r2 = r2_score(ytest, ypred_test)

print(f"MSE : {mse}")
print(f"RMSE : {rmse}")
print(f"MAE : {mae}")
print(f"R2 : {r2*100:.2f}%")
```

What this cell does

The code calculates Mean Squared Error, Root Mean Squared Error, Mean Absolute Error, and R^2 . The stored results are $MSE \approx 4059.85$, $RMSE \approx 63.72$, $MAE \approx 54.25$, and $R^2 = 26.66\%$.

ML concept / developer understanding

MSE and RMSE emphasize larger errors; MAE gives average absolute error; R^2 measures the proportion of target variance explained by the model relative to a constant baseline. In scikit-learn, the best possible R^2 is 1.0 and it can be negative. ■cite■turn0search0■

Stored notebook output

```
MSE : 4059.8526931097294
RMSE : 63.716973351766555
MAE : 54.25072743096797
R2 : 26.66%
```

Cell 36 – Feature analysis section heading

Cell type: markdown

Original notebook content

****Q9. Feature Analysis****

What this cell does

This markdown cell introduces feature analysis, but it contains no additional code.

ML concept / developer understanding

The notebook has already used correlation ranking to select BMI as the single feature.

Cell 37 – Notebook conclusion

Cell type: markdown

Original notebook content

****As R^2 Score is less than 80% we cannot consider this model for final prediction and deployment****

What this cell does

The notebook concludes that because the R^2 score is below 80%, the model should not be considered for final prediction and deployment.

ML concept / developer understanding

This is the notebook's stated conclusion. A developer note: an 80% R^2 cutoff is a project-specific rule, not a universal machine-learning requirement; model usefulness depends on business context, target variability, error costs, validation strategy, and comparison with alternative models.

Cell 38 – Empty code cell

Cell type: code

What this cell does

This cell contains no code and no stored output.

ML concept / developer understanding

No modeling action occurs in this cell.

2. Important Linear Regression Concepts

Concept	Simple explanation
Feature (X)	Input variable used to make predictions. In this notebook, X is only `bmi`.
Target (y)	Continuous value the model tries to predict. Here it is the Diabetes dataset `target`.
Simple Linear Regression	A regression model with one predictor. Conceptually: $\hat{y} = b_0 + b_1X$.
Intercept	The model's baseline predicted value when the feature is zero in the model's feature scale.
Slope / coefficient	How much the prediction changes for a one-unit increase in the feature, holding the model form fixed.
Train/Test split	Training data fits the model; test data estimates how well it generalizes to unseen examples.
MSE	Mean squared prediction error. Squaring makes large errors contribute more heavily.
RMSE	Square root of MSE, returning the error to the target's scale.
MAE	Mean absolute prediction error. Easier to interpret than MSE because it uses absolute differences.
R ²	Coefficient of determination. It compares the model with a constant-baseline predictor; 1.0 is the best possible

Official scikit-learn reference: `LinearRegression` fits an ordinary least-squares linear model, while its `predict` method uses the learned model to produce predicted values. [scikit-learn.org/en/stable/modules/linear_model.html#linear-regression](#)

3. Notebook Results and Interpretation

Metric / result	Notebook value	Interpretation
Dataset shape	442 × 11	442 observations, 10 features, 1 target.
Missing values	0 in every column	No missing values were found by the notebook's check.
Duplicates	0	No exact duplicate rows were found.
Top correlated feature	bmi (≈ 0.586)	BMI had the largest absolute correlation with target among the listed features.
Train/Test	353 / 89	80% training and 20% testing.
Intercept	≈ 152.795	Learned baseline term of the fitted one-feature model.
BMI coefficient	≈ 970.942	Learned slope for the dataset's scaled BMI feature.
MSE	≈ 4059.85	Squared-error metric; larger errors are weighted more.
RMSE	≈ 63.72	Typical error scale represented as the square root of MSE.
MAE	≈ 54.25	Average absolute prediction error.
R ²	26.66%	The model explains a relatively limited portion of target variance in this test split.

Important: The notebook says the model should not be used for final prediction/deployment because R² is below 80%. That is the notebook's stated rule. From an ML-developer perspective, 80% is not a universal cutoff; you should compare against business requirements and stronger baselines/models, and ideally use cross-validation before a deployment decision.

4. Interview-Ready Explanation

What did you do in this project?

I loaded the Diabetes dataset from scikit-learn, performed EDA, checked missing values and duplicates, analyzed correlations, selected BMI as the strongest correlated feature, split the data into training and testing sets, trained a Simple Linear Regression model, generated predictions, and evaluated it using MSE, RMSE, MAE, and R².

Why did you use BMI?

In the notebook, BMI had the highest absolute correlation with the target, approximately 0.586, so it was selected for the one-feature linear regression model.

Why do we split data?

We train the model on one portion of the data and test it on unseen data so we can estimate how well the model generalizes.

Why use a pipeline?

The pipeline keeps preprocessing organized and reusable. Here it contains median imputation, even though the EDA showed no missing values.

What does R² = 26.66% mean?

For this test split, the fitted one-feature model explains about 26.66% of the variation in the target relative to a constant baseline. It indicates that BMI alone is not sufficient to explain most of the target variation in this model.

5. ML Developer Notes

- The notebook correctly separates training and test data before fitting the preprocessing pipeline.
- The final model is intentionally simple because only BMI is selected as X.

- The Diabetes dataset features are represented on a standardized/scaled numerical scale; therefore, the coefficient should be interpreted in those feature units, not as an ordinary raw BMI-unit effect.
- The notebook uses a single train/test split. For a stronger model-selection decision, cross-validation would provide a more stable estimate.
- The notebook jumps from Q7 to Q9; there is no Q8 section in the provided cells. This PDF preserves that structure instead of silently renaming it.
- The final code cell is empty, so no additional operation is performed there.